AI Lawyer for refunds that never came — Kuku Problem 2

Mrityunjay Kumar · 22174016 · Engineering Physics · Prototype: https://github.com/mrityunjaykamlapuri007-coder/AI---Lawyer · Raw outputs: `ai_lawyer/evaluation/results/`

1. Problem and why this one

A refund that never arrives after a cancelled or returned online order is one of the most common legal problems an ordinary person in India has, and one of the least likely to reach a lawyer: the amount is small, the process is unclear, and the platform's support loop wears people down. I chose it because it is frequent, document-heavy, and governed by one short set of rules that can be verified end to end.

Scope (assumption): unresolved e-commerce refunds in India; English; text plus screenshots or PDFs as proof. Out of scope: voice, other languages, other disputes, Consumer Commission filing.

2. What it does for the user

The user chats normally and can attach a screenshot or PDF. The assistant (1) **understands the message**: greetings get a greeting, off-topic requests get "I can't help with that — I help when a refund goes wrong", details sent over several messages are combined, and each upload gets a verdict (✅ read, with what was found / ⚠️ not an order or refund record / ❌ unreadable); (2) **builds the case**: dates, amount, refund status, who was contacted, each fact tagged *user-reported* or *document-supported* and kept only if its words appear in the source; (3) **shows where they stand** with the relevant E-Commerce Rules 2020 provisions quoted word for word; (4) **recommends one proportionate next step** — wait (still inside the seller's own promise), contact support, formal grievance to the platform's grievance officer, or "I need one more fact" — with steps filled from their own facts; (5) **drafts the letter** with `[TO BE FILLED BY USER]` blanks. It never files anything, never says the seller "broke the law", and flags when a lawyer is needed.

3. How it works: what is AI, what is code, where the human is

```
message (+ file -> local OCR -> model document check) -> model router (greeting / off-topic / needs
details / refund case / follow-up) -> fact extraction -> retrieval (BGE dense + BM25, RRF)
 -> relevance grading + rule-based applicability -> Python outcome rules R0-R7 -> guards -> answer + draft
```

LLM (Qwen2.5-7B, swappable in `.env`)	Code	Human
routing each message, short chat replies, document check, fact extraction, relevance grading	corpus verification, retrieval, applicability, **choosing the outcome**, every guard, the summary, steps and draft	sends the letter, decides to escalate, sees a lawyer when flagged

Python owns the outcome. In a real run the model proposed "needs more information" for a case that clearly called for a grievance; the rules overrode it and logged the disagreement. **Live testing changed the design:** a 7B model sometimes added conditions that were not in the case when explaining a rule (e.g. "force majeure"). I removed model-written explanations: the answer now shows each rule's key clause verbatim, and Python builds the summary and steps from the extracted facts. Chat replies are checked too: any mention of a rule, deadline or number of days is replaced with a safe reply.

4. How it avoids invented law

- **Small verified corpus instead of model memory:** 30 provisions of the E-Commerce Rules 2020 and the 2021 amendment. Each source PDF is **byte-identical to the Gazette of India copy**; each provision's text matches an independent OCR reading word for word, numbers exact, with amendments applied at sub-rule level. Machine-verified — **not lawyer-reviewed**, and said so in the UI.
- **Citations by id, rendered from the corpus**, never from model text; unknown ids are blocked.
- **Any section number not in the corpus blocks the answer.** Result: **0** such numbers in 50 real-model answers.
- **Abstention:** no applicable verified provision → it says so and asks; it never fills the gap.

5. How I know it's good — and where it fails

Test cases: 17 real disputes restated from public reports of consumer-commission orders (anonymised, sourced, split 7 development / 10 test before any run; expected outcome = the rules applied to the stated facts) + 17 synthetic edge cases reported separately.

Run (real model on Kaggle)	Test (n=10)	Dev	Notes
1	crashed on case 5		4 defects: free-text status, negation, junk ids, amount crash
2	8/10	2/5	0 invented sections, 0 schema failures
3	6/17 overall		tunnel died; output cap; a regression from my own fix
4	**8/10**	**4/5**	final scored run
Held-out (4 unseen, pre-registered)	**3/4**		collected after the fixes

These scores were measured **before** the final chat changes (router, grounded answers, attachment checks). Those changes pass 381 offline tests and 18/18 fixture checks and were tried live on about a dozen conversations, but were not re-scored on the real-story set before the deadline.

Where it fails (fact extraction and scope, never invented law): a credited refund labelled "processed"; two events in one sentence swapping dates; a fix of mine that turned a delivery date into a purchase date (R16, reported, not hidden); a police complaint refused as "criminal" (H03, now routed by the model instead of a keyword); "last week" is not a date, so it asks for the exact one. Retrieval: recall@10 0.57 on machine-derived labels — a weak metric, and I say so.

6. Scale and cost

Per refund case: ~5 LLM calls, ~9,000 tokens, ~45 s on two free Kaggle T4s; greetings, off-topic messages and unusable files answer in 2–4 s. Cost at volume = tokens × the provider's price; the model is swapped in `.env` (any OpenAI-compatible API). Retrieval runs locally. Growing coverage means feeding new Gazette PDFs through the same verification pipeline, not longer prompts. First steps at

real volume: lawyer review of the corpus, and a hosted model with an uptime guarantee (the free Kaggle tunnel went down during my own testing; the app then falls back to rule-based replies).

7. Tools, and one place the AI got it wrong

ChatGPT: analysing the brief, architecture design (input pipeline, case matching, storage, legal RAG, verification, evaluation plan), reviewing my earlier research-RAG repo for reuse, and the v2 coding-agent spec. **Claude (chat):** cutting scope to what fits the deadline, instruction files for the coding agent, 17 synthetic behaviour cases, amendment research. **Claude Code:** implemented and tested the product in my repo. Runtime: Kaggle, vLLM, Qwen2.5-7B, BGE, Tesseract. The scope, models and final claims are my decisions, and I corrected the tools when they were wrong: ChatGPT first made "refer to a lawyer" the default outcome; I made the assistant assess the next step itself. **Where it got it wrong:** the coding agent wrote a 4-bit model id from memory that does not exist on Hugging Face; a check caught it before it broke the Kaggle run. In live testing, an uploaded project spec was read as a refund case because "SHA-256" looked like an order number — fixed by letting the model judge whether a file is an order or refund record before any fact is taken.

8. Limits and when to see a lawyer

Not lawyer-reviewed; no Consumer Protection Act yet, so it stops at the grievance stage; English text only; the document check says whether a file is readable and relevant, not whether it is genuine; local demo (no login, files stored unencrypted). It tells the user to see a lawyer for court or limitation deadlines, fraud, unclear liability between seller, platform and bank, or when they want representation.